of figure 2.9.B.

First, at line #2 we remove the last element of the heap, at index [8], and save it to a temporary variable; at line #3 we check if the remaining array is empty: if that had been the case, the last element in the array would also have been the top of the heap (its only element!) and we could just have returned it.

But this is clearly not the case, in this example, so we move on to line #4, where we store the first element of the heap, `<Unencrypted password on DB, 10>`, to a temporary variable, which will alter be returned by the method; at this point, we can imagine the heap like what's shown in figure 2.10.A, two disconnected branches without a root: to sew them together, we add a new root, using the element we had saved at line #2, the one entry that used to be the last in the array, at index `[8]`: figure 2.10.B shows the situation at this point.

This new root, however, is violating heap's properties, so we need to call `pushDown` at line #5 to reinstate them by swapping it with its second children, whose priority is `9.5`, and producing the heap shown in figure 2.10.C; method `pushDown` will handle this phase, stopping when the right position for element `<Memory Leak, 9>` is found.

To conclude this sub-section, it's worth noting that the `peek` method returns the top element in the heap, but it just returns its without any side effects on the data structure; its logic is therefore trivial, so we will skip its implementation.